Computer Science & IT



Data Structure & programming

Linked List

Lecture No. 05



By- Abhishek Sir

Recap of Previous Lecture



Topic	Doubly Linked List
Topic	getnode, build123()
Topic	Addend Addbegin
Topic	Addx befor y
Topic	

Slide

Topics to be Covered



Topic

Topic

Topic

Topic

Topic

Circular Linked List

Stack using Linked list

queue using linked list

Slide




```
dnode* Addx before y( dnode *s, int x, int y)
{
    dnode *q=s, *r=NULL,
    dnode *p=getnode(x);
    if(s==NULL){
        free(p);
        return s;
    }else if(s->data=y){
        p->next=s,
        s->prev=p
        return p,
    }
```

```
while(q->next!=NULL && q->data!=y)
    q=q->next,
if(q->data==y){
    r= q->prev,
    p->next = q;
    p->prev = r;
    q->prev = p,
    r->next = p,
}
else{
    printf("y is Not present");
    free(p),
}
return s;
}
```

Slide

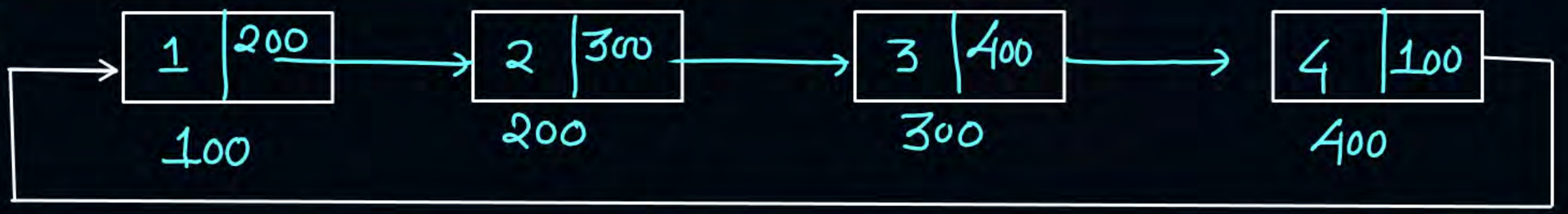



```
dnode* Addx before y( dnode *s, int x, int y)
{
    dnode *q=s, *r=NULL,
    dnode *p=getnode(x);
    if(s==NULL){
        free(p);
        return s;
    }else if(s->data==y){
        p->next=s,
        s->prev=p
        return p,
    }
```

```
while(q->next!=NULL && q->data!=y)
    q=q->next,
if(q->data==y){
    r= q->prev,
    p->next =q;
    p->prev = r;
    q->prev = p,
    r->next = p,
}
else{
    printf("y is Not present");
    free(p),
}
return s;
}
```

Slide

Topic : Circularlinked List



Circular Linked list there No NULL teominated Node

Last Node points to first Node.

Round Robin Algorithm

Slide

Topic : Circularlinked List



display value of circular list

```
display ( node *S) {
    while (S->next != HP) {
        printf ("%d", S->data);
        S = S->next.
    }
    printf ("%d", S->data);
}
```

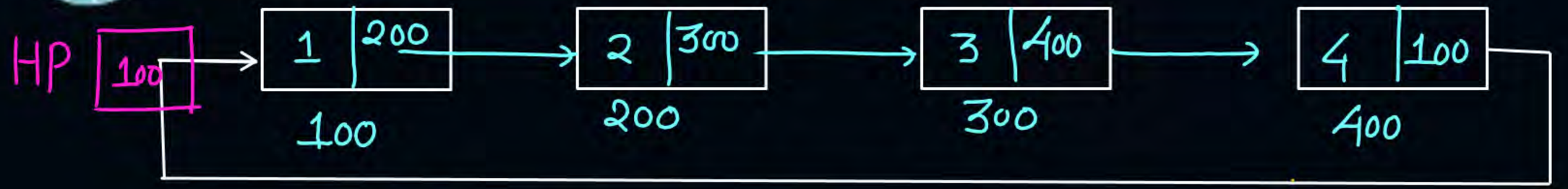
1 200 | = 100 | 3 00 | = 100 | 400 | = 100

1 | 2 | 3

100 | = 100 ← last value will not printed

Slide

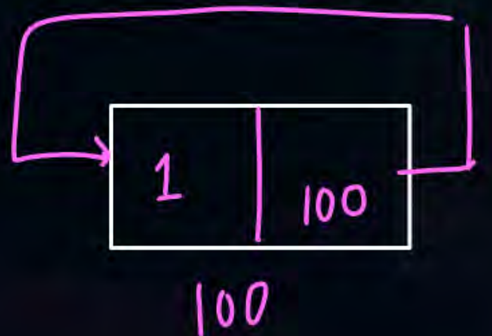
Topic : Circularlinked List




```
Void display (node *s) {
  if(s){
    do{
      printf("%d", s->data);
      s=s->next;
    }while(s!=HP);
  }
}
```

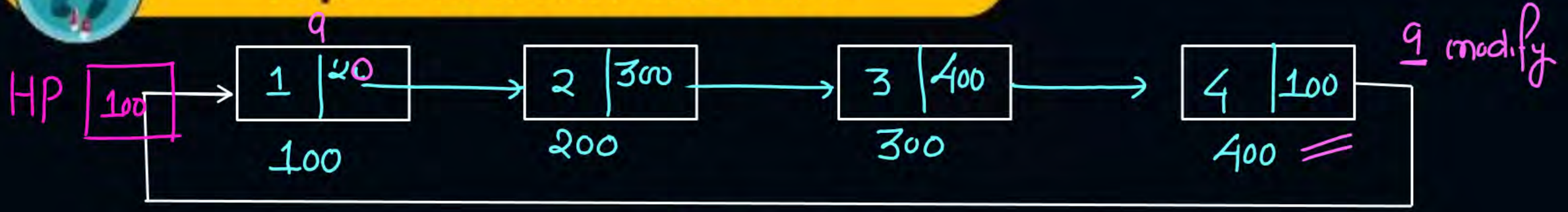
node *q=s;

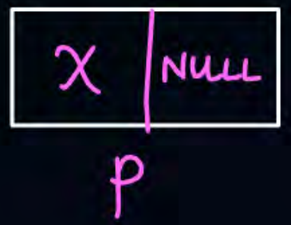
1	2	3	4
S=200	S=300	S=400	S=100
200!=100	300!=100	400!=100	100!=100



Slide

Topic : Circular linked List





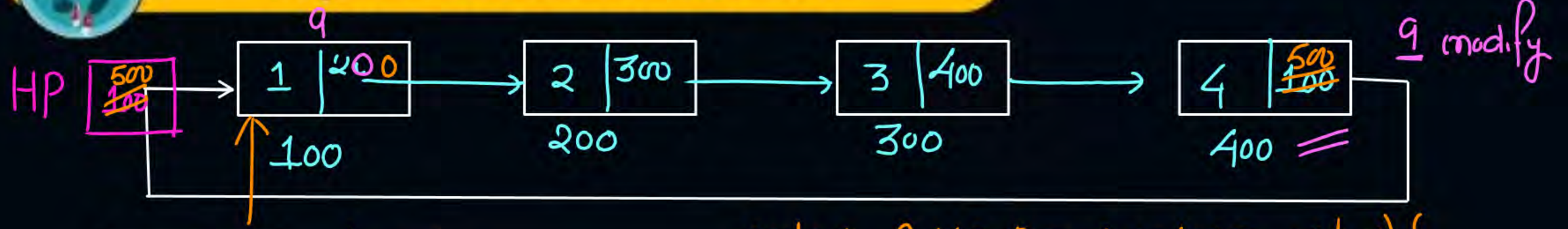
p→next = q

HP = p;

} Last Node will not modify

Slide

Topic : Circularlinked List





500 P

Local

HP ✓

Non-empty

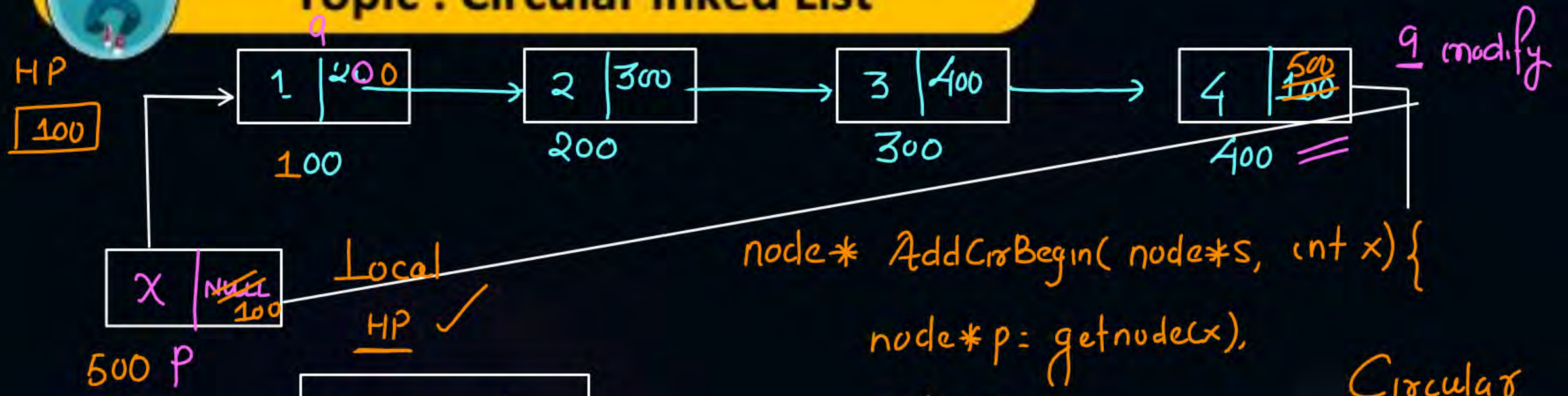
```
node* AddCirBegin(node*s, int x){
    node*p= getnode(x);
    node*q= s;
    while (q->next!=s)
        q=q->next;
    p->next=s;
    q->next=p;
    return p;}
```

Circular Single Linked Addbegin O(n)

Slide

Topic : Circular inked List





Non-empty

```
node* AddCirBegin(node*s, int x){
node*p= getnode(x);
node*q=s;
while(q->next!=s)
    q=q->next;
p->next=s;
q->next=p;
return s;
}
```

Circular Single Linked Addbegin O(n)

Slide

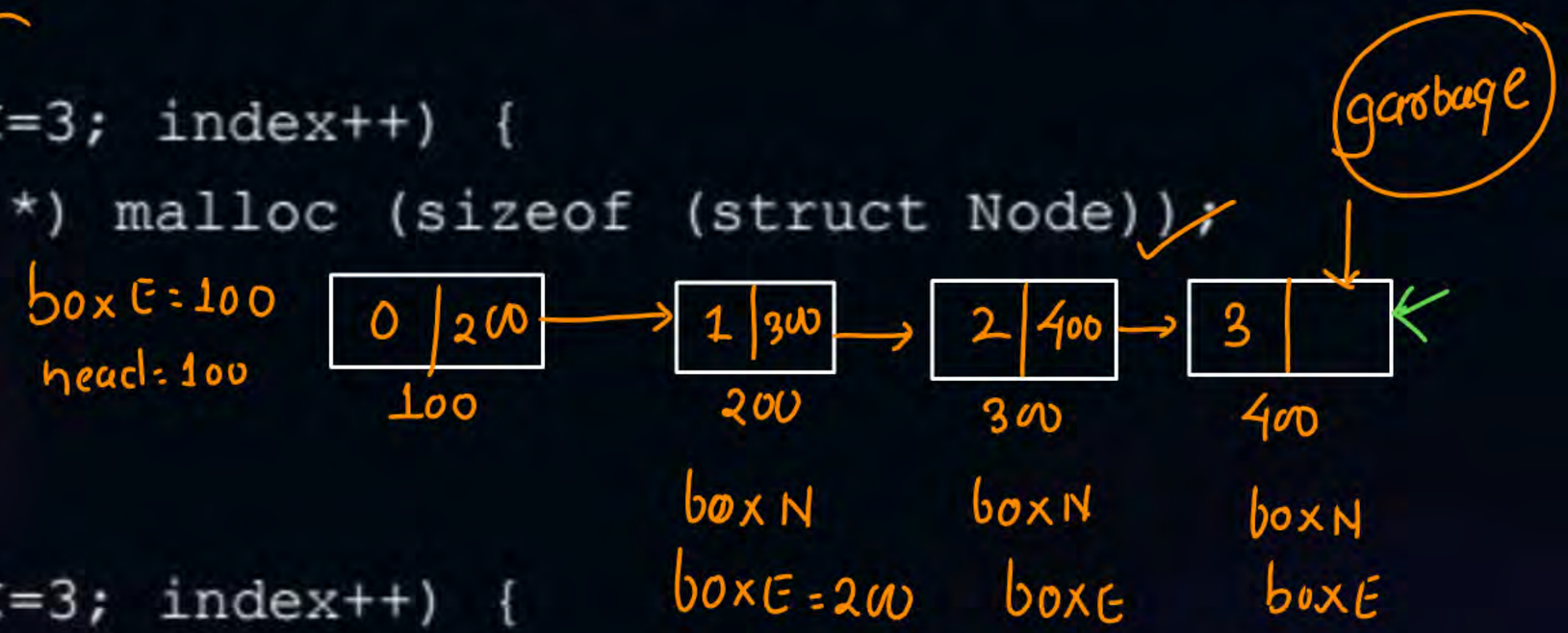
Consider the following ANSI C program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{
        int value;
        struct Node *next;};
```

Consider the following ANSI C program:

GATE 2021

```
int main () {
 Struct Node *boxE, *head, *boxN; int index = 0;
 boxE = head = (struct Node*) malloc (sizeof (struct Node));
 head->value = index;
 for (index = 1; index <=3; index++) {
   box N = (struct Node *) malloc (sizeof (struct Node));
   BoxE->next = boxN;
   boxN->value = index;
   boxE= boxN; }
 for (index = 0; index <=3; index++) {
      printf( Value at index %d is %d\n , index, head->value);
      head = head->next;
      printf ( Value at index %d is %d\n , index + 1, head- >value
 }}
```

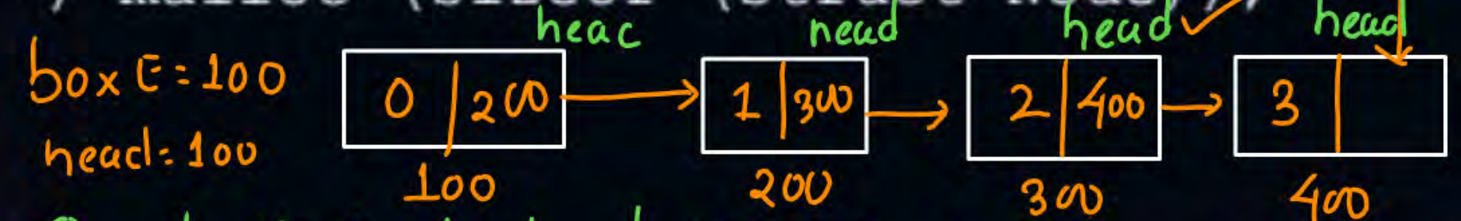


Consider the following ANSI C program:

GATE 2021

```
int main () {
 Struct Node *boxE, *head, *boxN; int index = 0;
 boxE = head = (struct Node*) malloc (sizeof (struct Node));
 head->value = index;
for (index = 1; index <=3; index++) {
  box N = (struct Node *) malloc (sizeof (struct Node));
  BoxE->next = boxN;
  boxN->value = index;
  boxE= boxN; }
for (index = 0; index <=3; index++) {
     printf( Value at index %d is %d\n , index, head->value);
     head = head->next;
     printf ( Value at index %d is %d\n , index + 1, head- >value
}}
```

garbage



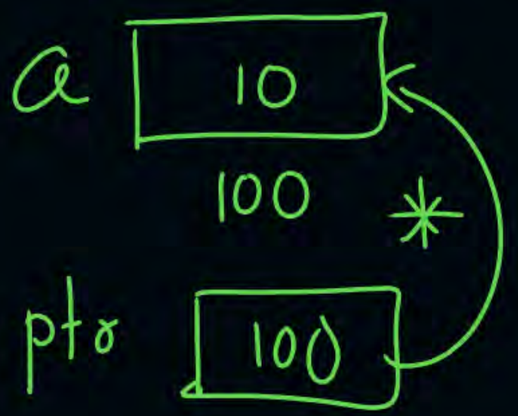
boxE: 100
head: 100

i=0 0,0 1 1 | i=1 1,1 2 2 | i=2 2,2 3 3 | i=3 3,3 — 4 —

runtime

```
int a = 10;
int *pto = &a;
```

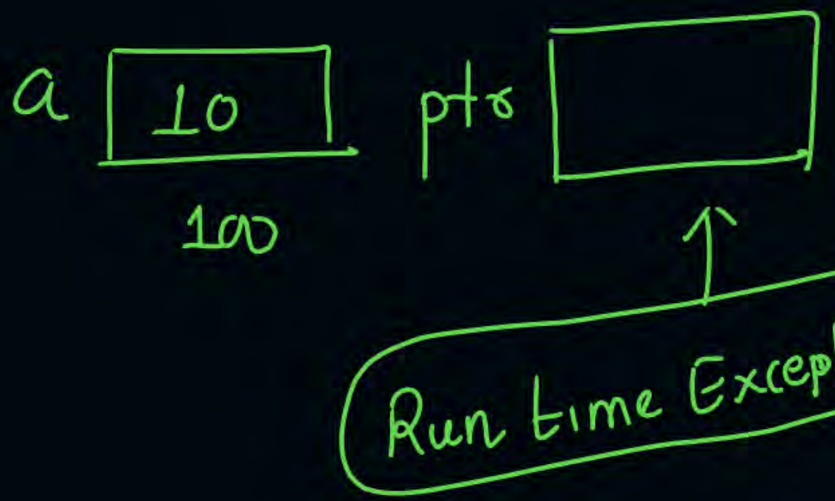
```
printf("%d", *pto);
```



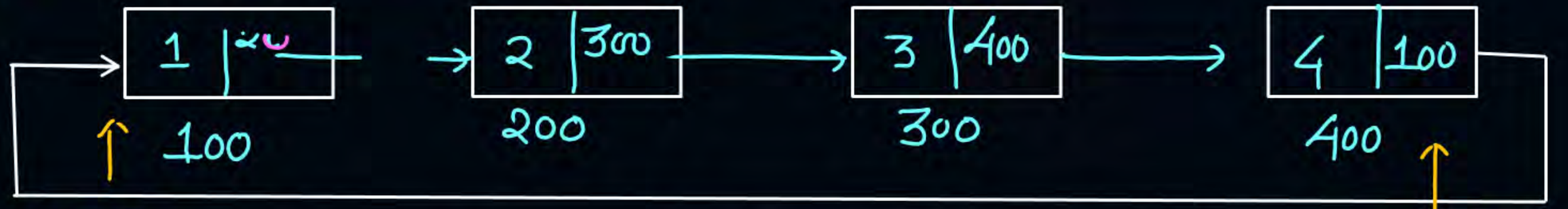

```
int a = 10;
int *pto;
```

← pointer Not initialized

```
printf("%d", *pto);
```



Topic : Circularlinked List



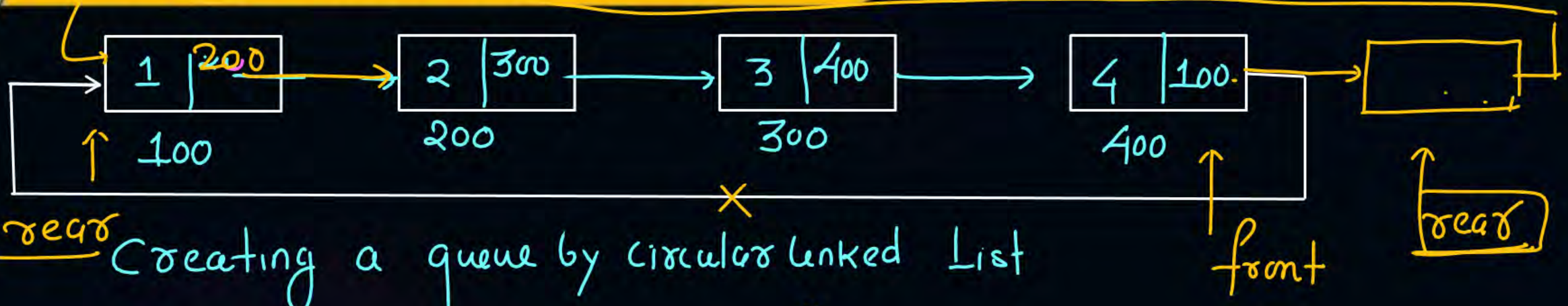
Creating a queue by circular linked List

Delete front
Insert rear
} panter

front next = rear (I)

Slide

Topic : Circularlinked List



Creating a queue by circular linked List

Delete front } panter
Insert rear }

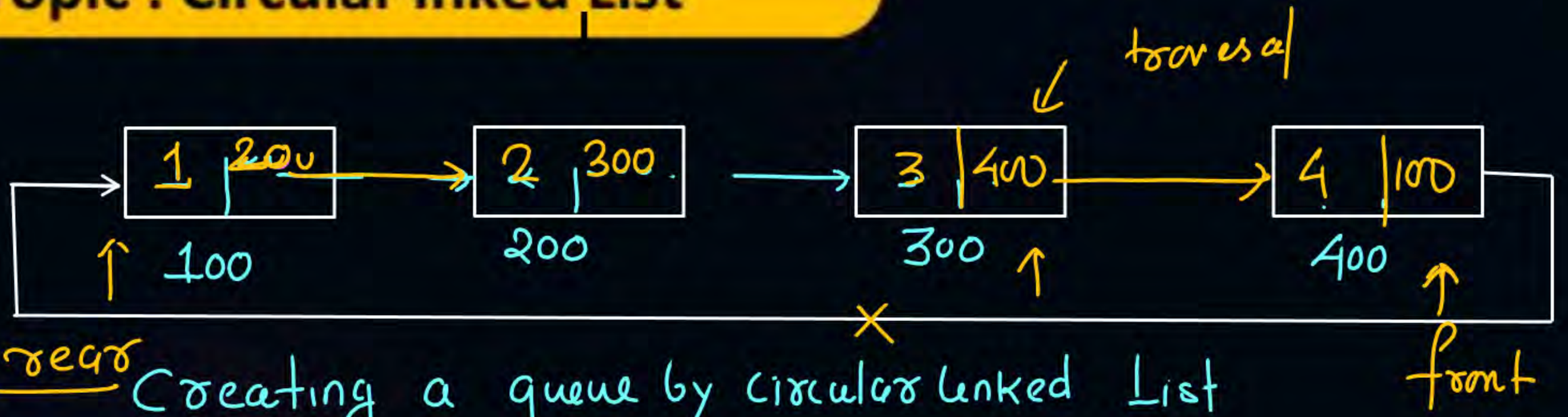
front next : rear (I)

Enqueue — Length dependend

New Node O(1)

Slide

Topic : Circular inked List



Creating a queue by circular linked List

Delete front } panter
Insert rear }

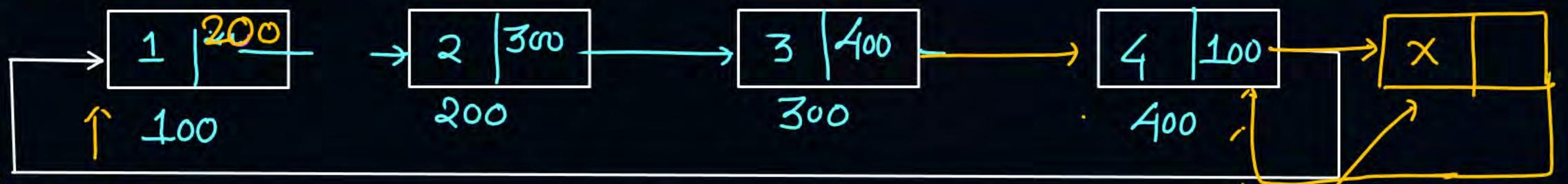
front next = rear (I)

dequeue ( ) O(n)

we can't go backward

Single linked list

Topic : Circularlinked List



Creating a queue by circular linked List

Delete front } panter
Insert rear }

rear → next = front

Dequeue : O(1) }
Enqueue : O(1) ✓ }

Slide

Topic : Circularlinked List



Creating a queue by circular linked List

Delete front } pointer
Insert rear }

rear → next = front

Dequeue : O(1)

Enqueue :

Slide

Which one of the statements below is correct about the program?

(A) It dereferences an uninitialized pointer that may result in a run-time error

(B) It has a missing return which will reported as an error by the compiler

(C) Upon execution, the program creates a linked-list of five nodes

(D) Upon execution, the program goes into an infinite loop



Question

A circular queue has been implemented using a single linked list where each node consists of a value and a single pointer pointing to the next node. We maintain exactly two external pointers **FRONT** and **REAR** pointing to the front node and the rear node of the queue, respectively. Which of the following statements is/are **CORRECT** for such a circular queue, so that insertion and deletion operation can be performed in O (1) time ?

I. Next pointer of front node points to the rear node.

II. Next pointer of rear node points to the front node.

(A) I only (B) II only

(C) Both I and II (D) Neither I nor II

Slide

Question

A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer, as shown in the figure. Let n denote the number of nodes in the queue. Let 'enqueue' be implemented by inserting a new node at the head, and 'dequeue' be implemented by deletion of a node from the tail.



Which one of the following is the time complexity of the most time-efficient implementation of 'enqueue' and 'dequeue, respectively, for this data structure?

A. $\Theta(1), \Theta(1)$
B. $\Theta(1), \Theta(n)$
C. $\Theta(n), \Theta(1)$
D. $\Theta(n), \Theta(n)$

Slide



Question

Consider the C program below

```
#include <stdio.h>
int *A, stkTop;
int stkFunc (int opcode, int val){
    static int size=0, stkTop=0;
    switch (opcode) {
        case -1: size = val; break;
        case 0: if (stkTop < size ) A[stkTop++]=val;
                break;
        default: if (stkTop) return A[--stkTop];
    }
    return -1;
```



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Circular queue

queue using Circular queue

- front → rear
  - Enqueue O(1)
  - dequeue O(n)
- rear → next
  - Enqueue O(1)
  - dequeue O(1)

Slide

THANK - YOU